RECAP Progress Report
Room 315 Kinematic Shuttle
Week of April 14–20, 2026

April 21, 2026

Project Overview

During this week, the Room 315 shuttle work became a structured kinematic simulation instead of an early proof of concept. The main idea is simple: measured CSV rail points provide the geometry, while an explicit rail network provides the routing logic. The shuttle does not rely on wheel–rail contact dynamics in this phase. Instead, its pose is computed along the current rail segment and sent to Gazebo with `set_pose`.

This approach made the system easier to test, easier to debug, and much more stable for demonstrations.

What Was Completed This Week

- Organized the Room 315 rail into 14 directed segments and 12 anchor nodes.
- Built `rail_network.yaml` with explicit switches, fixed transitions, stoppers, and start slots.
- Added CSV preprocessing, network validation, and debug plots.
- Implemented the kinematic shuttle core with explicit `MOVING`, `WAITING`, and `FALLING` modes.
- Added ROS 2 and Gazebo runtime integration for both Room 315 only and the full-floor world.
- Added independent switch commands, independent stopper commands, and sensor-style approach events.
- Added multi-shuttle support, runtime shuttle spawning, shuttle ON/OFF control, and start-slot occupancy checks.
- Added simple collision avoidance with a default distance of `0.33 m`.
- Added a continuous path backend using `cubic_hermite`, while keeping `polyline` as a reference backend for validation.

Main Problems and How They Were Solved

Problem	Solution
Endpoint matching was too fragile	Explicit nodes and routing tables were used instead of exact endpoint equality.
Multiple shuttles could overlap	Start-slot occupancy checks and collision avoidance were added.
Point-only paths were not the cleanest long-term model	A continuous `cubic_hermite` path backend was introduced and validated against the CSV polyline reference.

Current System Picture

At the end of the week, the system can:

- launch in Room 315 only or in the full-floor Gazebo world;
- start one or more shuttles from validated start slots;
- route shuttles through the rail network according to switch states;
- stop shuttles before switches with independent stopper commands;
- publish approach sensor events for teaching and testing;
- add new shuttles during runtime;
- enable or disable each shuttle independently;
- stop following shuttles before they overlap;
- use continuous path sampling while keeping the CSV files as the measured source of truth.

Validation Snapshot

The current network contains:

- 14 rail segments;
- 12 nodes;
- 4 switches;
- 4 stoppers;
- 4 validated start slots.

The continuous path validator finished with `PASS` and no warnings. The maximum deviation between the continuous backend and the polyline reference was about `0.00197 m`, which confirms that the new continuous representation remains very close to the measured path.

Figures

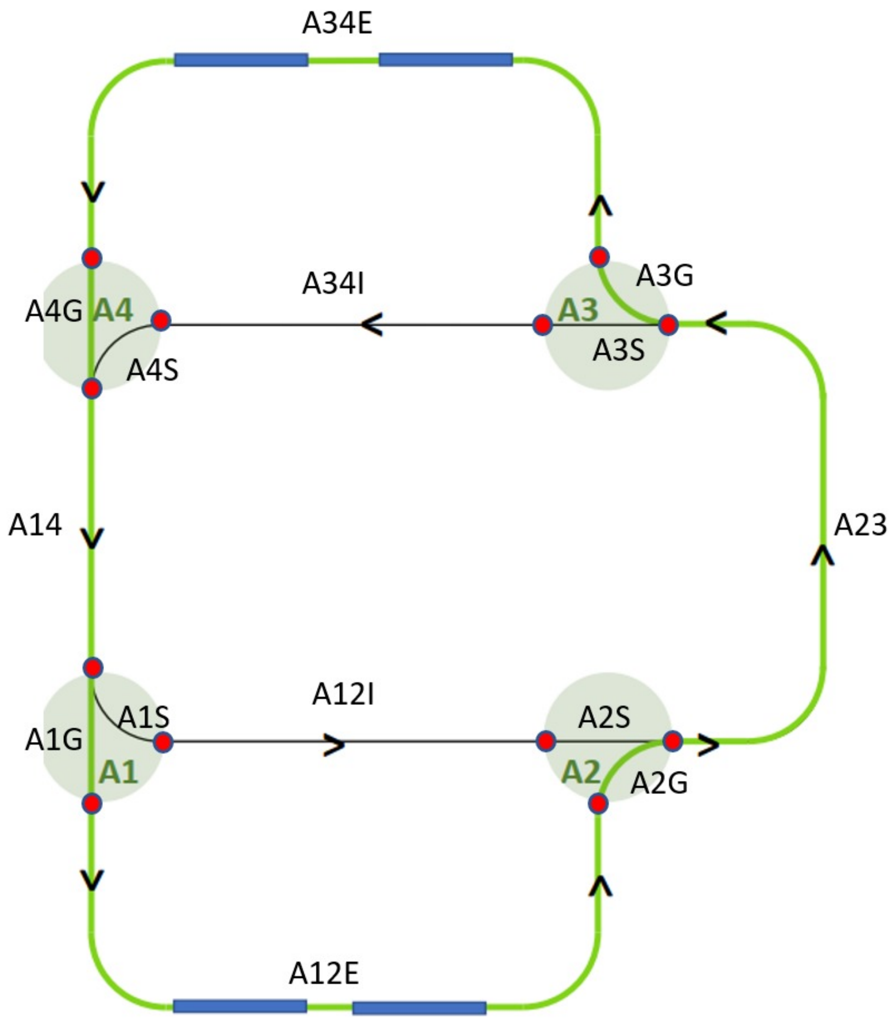

Figure 1: Conceptual Room 315 rail topology used as the basis for the software network model.

Executable routing table

Current segment	Rule type	Rule object	Result / behavior
`A14`	`switch_select`	switch `A1`	`G->A1G`, `S->A1S`, unknown state `->FALLING`.
`A1G`	`fixed`	–	Always continue to `A12E`.
`A1S`	`fixed`	–	Always continue to `A12I`.
`A12E`	`switch_guard`	switch `A2`	`G->A2G`, `S->FALLING`, unknown state `->FALLING`.
`A12I`	`switch_guard`	switch `A2`	`S->A2S`, `G->FALLING`, unknown state `->FALLING`.
`A2G`	`fixed`	–	Always continue to `A23`.
`A2S`	`fixed`	–	Always continue to `A23`.
`A23`	`switch_select`	switch `A3`	`G->A3G`, `S->A3S`, unknown state `->FALLING`.
`A3G`	`fixed`	–	Always continue to `A34E`.
`A3S`	`fixed`	–	Always continue to `A34I`.
`A34E`	`switch_guard`	switch `A4`	`G->A4G`, `S->FALLING`, unknown state `->FALLING`.

Figure 2: Executable routing table idea. Segment transitions are explicit and can lead to `FALLING` when the switch configuration is invalid.

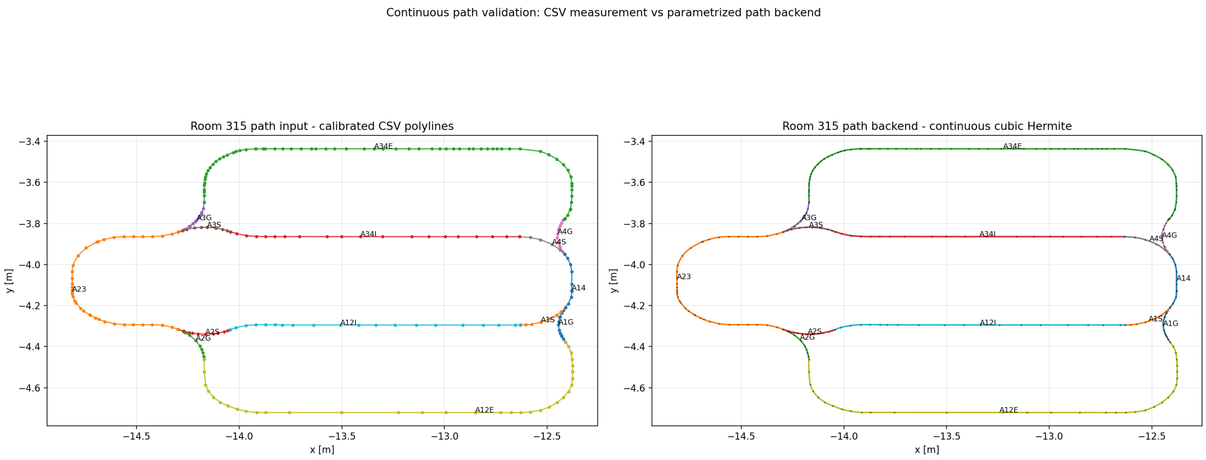

Figure 3: Continuous path validation. The `cubic_hermite` backend stays very close to the calibrated CSV polyline while providing smoother runtime sampling.

Next Step

- After finishing one side of the shuttles and sensors, replicate the same setup on the other side.
- Adjust the size of the cage.
- Rename the switch labels to `exterior` and `interior`.
- Make it possible to remove a shuttle completely from the simulation.
- If shuttles fall, make sure they can be reset without shutting down and restarting Gazebo.